



C++ DSA CHEAT SHEET

One Page • For Interviews & Competitive Programming



BASIC / I/O

```
#include <bits/stdc++.h>
using namespace std;
```

FAST I/O

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

INPUT / OUTPUT

```
int n; cin >> n;
cout << n << "\n";
```

DATA TYPES & MEMORY (64-bit)

Type	Size	Range (Approx.)
bool	1 byte	true / false
char	1 byte	-128 to 127
short	2 bytes	-32K to 32K
int	4 bytes	-2.1e9 to 2.1e9
unsigned int	4 bytes	0 to 4.29e9
long long	8 bytes	-9e18 to 9e18
float	4 bytes	~7 decimal digits
double	8 bytes	~15 decimal digits
long double	16 bytes	Higher precision

MEMORY QUICK REFERENCE (int)

10 ³ = 4 KB	10 ⁶ = 4 MB
10 ⁵ = 400 KB	10 ⁷ = 40 MB

STL CONTAINERS

Container	Description	Time Complexity (Avg.)
vector<T>	Dynamic array	push_back O(1) access O(1)
string	Sequence of chars	O(1) access
deque<T>	Double ended queue	push/pop O(1)
list<T>	Doubly linked list	insert/delete O(1)
stack<T>	LIFO	push/pop O(1)
queue<T>	FIFO	push/pop O(1)
priority_queue<T>	Max Heap (default)	push/pop O(log n)
set<T>	Ordered (RB Tree)	O(log n)
unordered_set<T>	Hash Set	O(1)
map<K,V>	Ordered Map	O(log n)
unordered_map<K,V>	Hash Map	O(1)

COMMON OPERATIONS

```
sort(v.begin(), v.end())
reverse(v.begin(), v.end())
find(v.begin(), v.end(), x)
count(v.begin(), v.end(), x)
min(a, b) / max(a, b)
swap(a, b)
*min_element(v.begin(), v.end())
*max_element(v.begin(), v.end())
accumulate(v.begin(), v.end(), 0LL)
iota(v.begin(), v.end(), 0) // 0 1 2 ... n-1
```

COMPLEXITY

O(n log n)
O(n)
O(n)
O(n)
O(1)
O(1)
O(n)
O(n)
O(n)
O(n)

LOOPS

```
for (int i = 0; i < n; i++) { }
for (int i = n-1; i >= 0; i--) { }
for (auto x : v) { }
while (condition) { }
```

FUNCTIONS

```
int add(int a, int b) {
    return a + b;
}
void solve(vector<int> &v) { }
// pass by reference to avoid copy
```

PAIR

```
pair<int,int> p = {1,2};
p.first // 1
p.second // 2
```

VECTOR

```
vector<int> v;
v.push_back(x);
v.pop_back();
v.size();
v.empty();
v.clear();
v[i];
v.front(); v.back();
sort(v.begin(), v.end());
reverse(v.begin(), v.end());
lower_bound(v.begin(), v.end(), x)
upper_bound(v.begin(), v.end(), x)
```

STRING

```
string s;
s.size();
s.empty();
s.push_back(ch);
s.pop_back();
s.substr(i, len);
sort(s.begin(), s.end());
reverse(s.begin(), s.end());
s.find("pattern");
s.erase(i, len);
```

BIT MANIPULATION

```
x & y // AND
x | y // OR
x ^ y // XOR
~x // NOT
x << 1 // Left shift
x >> 1 // Right shift
x & (x-1) // Clear last set bit
```

BUILTIN FUNCTIONS

```
__builtin_popcount(x) // set bits in int
__builtin_popcountll(x) // set bits in long long
__builtin_clz(x) // leading zeros
__builtin_ctz(x) // trailing zeros
```

LAMBDA (Custom Comparator)

```
sort(v.begin(), v.end(),
[]){int a, int b){
    return a > b; // descending
});
```

CONSTANTS

```
INT_MAX INT_MIN
LLONG_MAX LLONG_MIN
const int MOD = 1e9 + 7;
```

STL CONTAINER DETAILS

vector<T>	deque<T>	list<T>	stack<T>	queue<T>	priority_queue<T>	set<T>	unordered_set<T>	map<K,V>	unordered_map<K,V>
• Dynamic array	• Double ended queue	• Doubly linked list	• LIFO	• FIFO	• Max Heap by default	• Ordered (RB Tree)	• Hash Set	• Ordered Map	• Hash Map
• Contiguous memory	• push_front(x)	• insert(), erase()	• push(x)	• push(x)	• Min Heap:	• s.insert(x)	• insert, erase, find	• m[key] = val	• m[key] = val
• v.push_back(x)	• push_back(x)	• push_front(x)	• st.push()	• q.push()	• priority_queue<T, vector<T>, greater<T>>	• s.erase(x)	• Average O(1)	• m.find(key)	• find, erase
• v.pop_back()	• pop_front()	• push_back(x)	• st.pop()	• q.pop()	• pq.push(x)	• s.find(x)	• No order	• m.erase(key)	• Average O(1)
• v.size(), v.empty()	• pop_back()	• Not random access	• st.top()	• q.front()	• pq.pop(), pq.top()	• s.lower_bound(x)		• m.lower_bound(key)	• No order
• v[i] O(1)	• access O(1)		• st.empty()	• q.back()		• s.upper_bound(x)			

ARRAY

```
int arr[100];
arr[i];
sizeof(arr) // total bytes
sizeof(arr)/sizeof(arr[0]) // size
```

2D ARRAY

```
int a[n][m];
a[i][j];
Row-major order
```

TIPS

- Prefer long long for large values.
- Use '\n' instead of endl for speed.
- Always check edge cases.
- Dry run before coding.
- Think: Brute → Optimize → Code.

COMMON HEADERS

```
<vector> // vector
<string> // string
<queue> // queue, priority_queue
<stack> // stack
<map> // map, multimap
<set> // set, multiset
<unordered_map> // unordered_map
<unordered_set> // unordered_set
<algorithm> // sort, reverse, etc.
<numeric> // accumulate, iota
```

MATH

```
abs(x)
sqrt(x)
pow(a, b)
gcd(a, b) // __gcd(a,b)
lcm(a, b) // a / gcd(a,b) * b
```